Nesne Yönelimli Programlama

Çıkmış Vize Soruları ve Ayrıntılı Çözümleri

(Türkçe çözümler)

Sorular: sınav fotoğrafları, docx/md dosyaları, WhatsApp aktarımları
Çözümler: slaytlar, ders kitabı ve hocanın derste anlattıkları

9 Kasım 2026

Bu dosya hakkında

- Klasördeki bütün kaynaklarda bulunan **vize** sorularını içerir. Kaynak ve güvenilirlik her sınavın başında belirtilmiştir: *kâğıt fotoğrafı*, *docx/md*, *öğrenci aktarımı*. Final soruları (design pattern, SOLID kodlama) vize kapsamında olmadığı için alınmadı; yalnızca vize konularıyla örtüşenler alıştırmalarda kullanıldı.
- Vize kapsamı: **Chapter 1, 2, 3, 7, 11, 13** ve UML sunumu (WhatsApp: *"1 2 3 7 11 13. üniteleri işledi hepsinden sorumluyuz"*). Hafta 8 videosu klasörde yok; Hafta 9'un son iki parçası (UML) hocanın *"sınavımız bugünkü konulara kadar"* dediği kısımdır ve dahildir. Hafta 10'dan sonrası (C# uygulamaları) final konusudur.
- Sınav biçimi: **4–5 klasik soru**, ezbere ve çizime dayalı (ISO tablosu, stable intermediate forms, cohesion/coupling kodu, use case / class / sequence diyagramı). Cevaplar İngilizce terimlerle yazılmalı.
- **Dil:** Bu sürümde İngilizce cevapların **Türkçe** karşılıkları verildi. Türkçe yalnızca anlamak için; sınavda cevapları ve terimleri **İngilizce** yazın. İngilizce sürüm: `02_Cikmis_Sorular.pdf` (sitede **EN | TR** düğmesi).
- Konu dağılımı ve tahmin: `03_Soru_Analizi.pdf`.

İçindekiler

1 2025–2026 Güz Vizesi (öğrenci aktarımı, 5 soru) 2

1.1 Soru 1 – ISO/IEC 25010 (18 puan) . . . 2
1.2 Soru 2 – Stable Intermediate Forms . . . 2
1.3 Soru 3 – Cohesion / coupling türü ve düzeltilmesi . . . 3
1.4 Soru 4 – Rent a car otomasyonu: sınıf ilişkileri . . . 4
1.5 Soru 5 – Rent a car: sequence diyagramı (24 puan) . . . 5

2 2024–2025 Güz Vizesi (öğrenci aktarımı + hocanın çözümü) 6

2.1 Soru 1 – Stable Intermediate Forms (15 puan) . . . 6
2.2 Soru 2 – ISO/IEC 25010 tablosu (komple) . . . 7
2.3 Soru 3 – E-ticaret use case . . . 8

2.4 Soru 4 – Cohesion/coupling: tür ve daha iyi hâle getirme 9

3 Geçmiş yıl sınavı: sınıf diyagramındaki ilişkileri yorumlama (fotoğraf, yılı belirsiz) 10

4 Online dönem test vizesi (çoktan seçmeli, docx) 11

5 2014–2015 Vize I (sınav kâğıdı fotoğrafı, Bahriye Akay, 27.11.2014) 13

5.1 Soru 1 – Unified Process (15 p) 13

5.2 Soru 2 – IEEE SPMP başlıkları (15 p) 14

5.3 Soru 3 – Cohesion ve coupling (15 p) 14

5.4 Soru 4 – Singleton pattern (15 p) 15

5.5 Soru 5 – Proje takip sistemi (40 p) 16

6 Diğer çıkmış ve ödev soruları 17

6.1 Sınıflar arası ilişki türleri 17

6.2 Adres defteri senaryosu 19

6.3 2021–22 “2. Vize Projesi”: Üniversite kütüphane platformu 21

7 Klasik çıkmış kısa sorular bankası 23

8 Alıştırmalar: koddan cohesion ve coupling bulma 28

1 2025–2026 Güz Vizesi (öğrenci aktarımı, 5 soru)

Kaynak

Soru metni yok; WhatsApp grubundaki aktarımlardan derlendi: *"1. Stable intermediate forms 2. ISO maddeleri 3. cohesion coupling çeşitleri ve düzeltilmesi 4. rent a car otomasyonu okların çeşitleri (aggregation, composition, inheritance vs) ve işlevleri 5. Otomasyonun sequence diyagramı."* Puanlar: *"İlk soru 18 puan, ana başlıklar 1'er puan, her alt başlık 0.5 puan"* (ISO sorusu), *"son soru sequence diyagramı 24 puanmış"*. Cohesion sorusundaki kod: `public void kdvekle_indirimuygula(int indirimOrani)`: indirim oranına göre toplam tutardan indirim düşüyor, sonra KDV ekliyor; öğrencilerin hatırladığı ikinci kodda `faiz = faizOrani / 100` vardı ve "tüm işlemler aynı class'ta" yapılıyordu. Aşağıdaki kodlar bu aktarımlara göre **yeniden kurulmuştur**.

1.1 Soru 1 – ISO/IEC 25010 (18 puan)

Soru 1

ISO/IEC 25010 yazılım ürün kalitesi modelinin karakteristiklerini ve alt karakteristiklerini yazınız.

Çözüm.

Ana başlık 1 puan, alt başlık 0,5 puan: 9 ana başlığın her birinden en az **iki** alt başlık yazılırsa tam puan. Güvenli olmak için hepsini yazın.

Karakteristik	Alt karakteristikler
Functional Suitability (İşlevsel uygunluk)	functional completeness, functional correctness, functional appropriateness *(işlevsel tamlık, işlevsel doğruluk, işlevsel yerindelik)*
Performance Efficiency (Performans verimliliği)	time behaviour, resource utilization, capacity *(zaman davranışı, kaynak kullanımı, kapasite)*
Compatibility (Uyumluluk)	co-existence, interoperability *(birlikte var olabilme, birlikte çalışabilirlik)*
Interaction Capability (Etkileşim yeteneği)	appropriateness recognizability, learnability, operability, user error protection, user engagement, inclusivity, user assistance, self-descriptiveness *(uygunluğun tanınabilirliği, öğrenilebilirlik, çalıştırılabilirlik, kullanıcı hatasından koruma, kullanıcı katılımı, kapsayıcılık, kullanıcıya yardım, kendini açıklayabilirlik)*
Reliability (Güvenilirlik)	faultlessness, availability, fault tolerance, recoverability *(hatasızlık, erişilebilirlik (hazır bulunma), hata toleransı, kurtarılabilirlik)*
Security (Güvenlik)	confidentiality, integrity, non-repudiation, accountability, authenticity, resistance *(gizlilik, bütünlük, inkâr edilemezlik, hesap verebilirlik, kimlik doğruluğu, saldırıya direnç)*
Maintainability (Bakım yapılabilirlik)	modularity, reusability, analysability, modifiability, testability *(modülerlik, yeniden kullanılabilirlik, analiz edilebilirlik, değiştirilebilirlik, test edilebilirlik)*
Flexibility (Esneklik)	adaptability, scalability, installability, replaceability *(uyarlanabilirlik, ölçeklenebilirlik, kurulabilirlik, yer değiştirebilirlik)*
Safety (Emniyet)	operational constraint, risk identification, fail safe, hazard warning, safe integration *(operasyonel kısıt, risk tanımlama, arızada güvenli duruma geçme, tehlike uyarısı, güvenli bütünleşme)*

Türkçe karşılıklar yalnızca anlamak içindir; sınavda maddeleri **İngilizce** yazın. Açıklama istenmedi (hoca: "açıklamadan ziyade maddeler"). 2011 sürümünü bilen biri *Usability* ve *Portability* yazarsa yine puan alabilir, ama hoca 2023 tablosunu kullanıyor.

1.2 Soru 2 – Stable Intermediate Forms

Soru 2

Stable intermediate forms nelerdir? Açıklayınız.

Çözüm.

Karmaşık sistemler, kararlı ara formlardan (stable intermediate forms) geçerek gelişirlerse çok daha hızlı ve güvenli gelişir. Her adımda kararlı, çalışan bir form vardır; bir değişiklik başarısız olursa her şeye baştan başlamak yerine son kararlı forma geri dönülür. Bu, moving target probleminin ve kaçınılmaz hataların etkisini azaltır. Üç yolu vardır:

1. **Three points of contact (üç temas noktası):** *Üç uzvunu sabit tutup yalnızca birini hareket ettiren bir tırmanıcı gibi, aynı anda yalnızca tek bir şeyi değiştirir, diğer her şeyi sabit tutarız. Değişiklik test edilip kararlı hâle gelince sıradakini değiştiririz. Değişiklik olumsuz sonuç verirse geri dönmek kolaydır ve sorunun hangi değişiklikten çıktığını biliriz.*
2. **Copy before release (yayından önce kopyala):** *Yeni bir sürüm yayımlanmadan (değişiklik canlıya alınmadan) önce mevcut kararlı sürümün kopyası/yedeği alınır; yeni sürüm başarısız olursa kararlı kopyaya geri dönülür (baseline, sürüm kontrolü).*
3. **Iterative and incremental software development (yinelemeli ve artırımlı geliştirme):** *Ürün artırımlar (mini projeler) hâlinde geliştirilir ve her artırımda iş akışları yinelenir. Her artırımın sonunda çalışan ama eksik bir ürün, yani kararlı bir ara form vardır. En önemli çekirdekten başlanır (stepwise refinement, Miller yasası) ve ürün kartopu gibi büyütülür (snowball effect); riskler azalır, ürün yeni bilgilerle evrilir.*

1.3 Soru 3 – Cohesion / coupling türü ve düzeltilmesi

Soru 3 (yeniden kurulmuş kod)

Aşağıdaki modüllerin cohesion türlerini, modüller arasındaki coupling türünü bulunuz ve kodu daha iyi cohesion ve coupling'e sahip olacak şekilde yeniden yazınız.

```
class Banka
{
    public static double toplamTutar;           // global veri
    public void kdvekle_indirimuygula(int indirimOrani)
    {
        toplamTutar = toplamTutar - toplamTutar * indirimOrani / 100.0;
        toplamTutar = toplamTutar + toplamTutar * 0.20;
    }
    public void Islemler(Musteri m, double faizOrani)
    {
        double faiz = faizOrani / 100;
        Console.WriteLine("Hos geldiniz " + m.Ad);
        m.Bakiye = m.Bakiye + m.Bakiye * faiz;
        LogDosyasiniAc();
    }
}
```

Çözüm.

a) `kdvekle_indirimuygula`: communicational cohesion. *Metot iki eylem yapıyor (indirim uygulama ve KDV ekleme). Bu eylemler ürünün prosedürüyle ilişkili (önce indirim, sonra KDV) ve ikisi de aynı veri (toplamTutar) üzerinde çalışıyor; bu yüzden communicational cohesion'a sahiptir.* Öğrenciler procedural, logical, temporal da yazdı; "aynı veri" ölçütü communicational'a götürür. Gerekçeyi yazmak puanı kurtarır.

b) `Islemler`: coincidental cohesion. Faiz hesaplama, karşılama mesajı yazdırma ve log dosyası açma birbiriyle ilgisiz işlerdir; aynı metotta olmaları tesadüftür. (Hepsi oturum başlangıcında art

arda yapılıyorsa *temporal* demek de savunulabilir; hoca benzer örnekte "coincidental ya da temporal" dedi.)

c) Coupling:

- `toplamTutar public static`: bu alanı yazan her modül ile `kdvekle_indirimuygula` arasında **common coupling** vardır.
- `Islemler`'e bütün `Musteri` nesnesi gönderiliyor ama sadece `Ad` ve `Bakiye` kullanılıyor ⇒ çağıran ile `Islemler` arasında **stamp coupling**.

d) Düzeltilmiş kod: her metot tek iş yapar (functional), veri gizlenir ve işlemleriyle aynı sınıfta durur (informational), parametreler basit veri (data coupling).

```
class Sepet                                    // informational cohesion
{
    private double toplamTutar;                // information hiding, global yok
    public Sepet(double tutar) { toplamTutar = tutar; }
    public void IndirimUygula(int indirimOrani)          // functional
    { toplamTutar -= toplamTutar * indirimOrani / 100.0; }
    public void KdvEkle(double kdvOrani)                 // functional
    { toplamTutar += toplamTutar * kdvOrani; }
    public double Tutar() { return toplamTutar; }
}

class Hesap
{
    private double bakiye;
    public void FaizEkle(double faizOrani)               // functional, data coupling
    { bakiye += bakiye * faizOrani / 100; }
}

class Ekran   { public void Karsila(string ad) { Console.WriteLine("Hos geldiniz " + ad); } }
class Logger  { public void DosyayiAc() { /* ... */ } }
```

Yeniden düzenlemeden sonra her metot functional cohesion'a, her sınıf informational cohesion'a sahiptir ve bütün modüller data coupled'dır (yalnızca basit veri gönderilir, global veri yoktur).

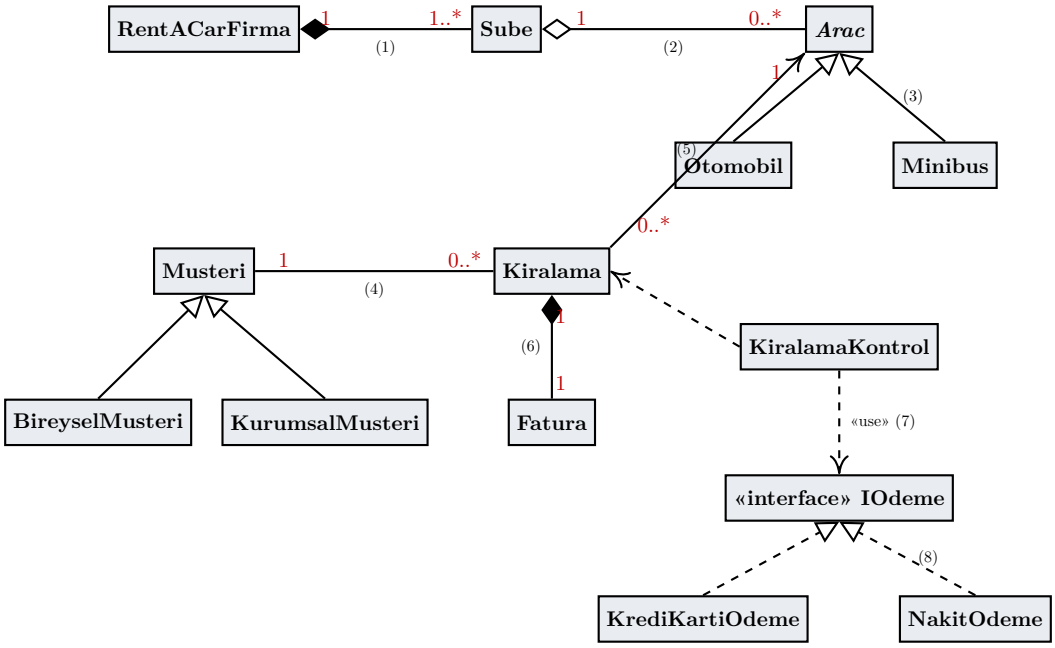
1.4 Soru 4 – Rent a car otomasyonu: sınıf ilişkileri

Soru 4 (senaryo öğrenci aktarımından)

Bir araç kiralama (rent a car) firmasının otomasyonu için sınıf diyagramı veriliyor/isteniyor. Diyagramdaki okların türlerini (association, aggregation, composition, inheritance, realization, dependency) ve senaryodaki sınıflar açısından işlevlerini açıklayınız.

Çözüm.

Sınav kâğıdındaki diyagram elimizde yok; aşağıdaki diyagram aynı senaryo için bütün ilişki türlerini içerecek şekilde kurulmuştur. Cevapta her ok için **tür + iki sınıf açısından anlam + çokluk** yazılmalıdır.



#	İlişki türü	Açıklama
1	Composition	Firma ♦ Şube: bir firmanın bir veya daha fazla şubesi vardır; şubenin yaşam döngüsünü firma kontrol eder, firma kapanırsa şubeler de yok olur; bir şube tek bir firmaya aittir.
2	Aggregation	Şube ◊ Araç: şube bir araç topluluğu içerir (0..*); araç şubeden bağımsız yaşar, başka şubeye transfer edilebilir. Elmas **Şube** tarafında.
3	Inheritance	Otomobil ve Minibüs *is-a* Araç: plaka, marka, günlük ücret gibi ortak özellikleri Araç'tan miras alırlar. Araç italik ⇒ abstract. Bireysel/Kurumsal Müşteri de Müşteri'den kalıtılır.
4	Binary association	Müşteri – Kiralama: ikisi birbirinden haberdar; bir müşterinin sıfır veya daha fazla kiralaması olabilir, her kiralama tek müşteriye aittir.
5	Directed association	Kiralama → Araç: kiralama aracın metotlarını çağırır (`musaitMi()`, `durumGuncelle()`), araç kiralamadan haberdar değil; her kiralama tam bir araca aittir, bir araç zamanla birçok kiralamada yer alır.
6	Composition	Kiralama ♦ Fatura: fatura kiralama tarafından oluşturulur ve kiralama olmadan var olamaz (1–1).
7	Dependency	KiralamaKontrol - - «use» > IOdeme: kontrol sınıfı ödemeyi geçici olarak kullanır (parametre); ödeme arayüzü değişirse kontrol etkilenir. Aynı şekilde KiralamaKontrol Kiralama'yı kullanır.
8	Realization	KrediKartiOdeme ve NakitOdeme IOdeme arayüzünü gerçekleştirir (*can-do*); yeni ödeme türü eklemek mevcut kodu değiştirmez (Open-Closed).

Sınıf türleri: Musteri, Arac, Kiralama, Fatura, Sube **entity**; KiralamaKontrol **control**; kiralama ekranı ve fatura çıktısı **boundary** olur.

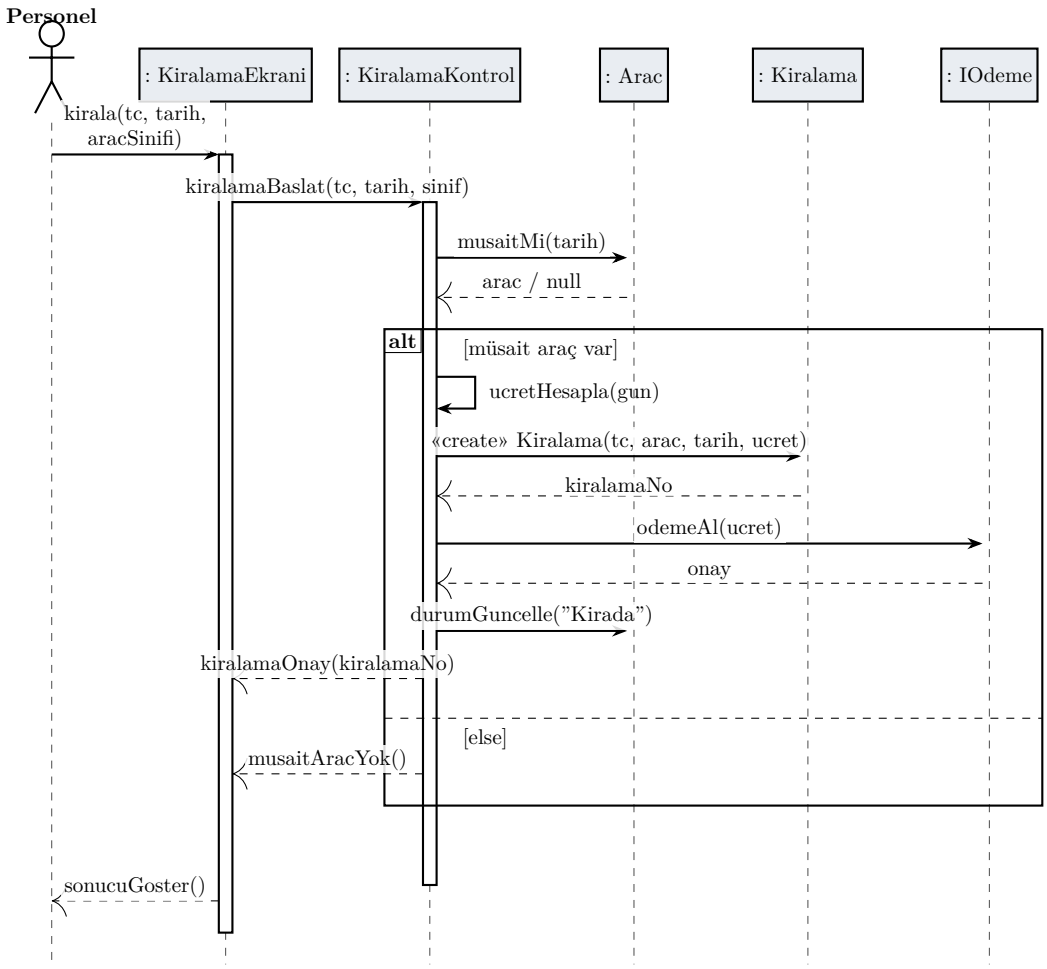
1.5 Soru 5 – Rent a car: sequence diyagramı (24 puan)

Soru 5

Rent a car otomasyonunun "Araç Kirala" senaryosu için sequence diyagramını çiziniz.

Çözüm.

Senaryo: Personel müşterinin bilgisini ve istediği tarih aralığını girer; sistem müsait araç arar; araç **varsa** ücret hesaplanır, kiralama oluşturulur, ödeme alınır, araç "kirada" olarak işaretlenir ve onay gösterilir; araç **yoksa** "müsait araç yok" mesajı gösterilir.



Puan kazandıran noktalar: (1) aktör + boundary + control + entity nesneleri doğru sırada; (2) her mesaj **okun ucundaki sınıfın metodu** ve parametreli yazılmış; (3) dönüşler kesikli; (4) aktivasyon kutuları; (5) if/else için `alt` çerçevesi; (6) self call (`ucretHesapla`); (7) akış use case adımlarıyla tutarlı.

2 2024–2025 Güz Vizesi (öğrenci aktarımı + hocanın çözümü)

Kaynak

`Çıkmışlar/2024-2025 Vize Soruları.md`, WhatsApp aktarımları (*"geçen sene vizede Stable Intermediate Forms konusunu açıklamalarıyla, ISO tablosunu komple, cohesion-coupling kodu türü ve düzeltimi bir de use case çıkmıştı"*) ve hocanın vizeden sonra tahtada gösterdiği use case çözümünün fotoğrafları.

2.1 Soru 1 – Stable Intermediate Forms (15 puan)

Soru 1

Stable intermediate forms'u açıklayınız.

Çözüm.

Karmaşık sistemler, kararlı ara formlardan (stable intermediate forms) geçerek gelişirlerse çok daha hızlı ve güvenli gelişir. Her adımda kararlı, çalışan bir form vardır; bir değişiklik başarısız olursa

her şeye baştan başlamak yerine son kararlı forma geri dönülür. Bu, moving target probleminin ve kaçınılmaz hataların etkisini azaltır. Üç yolu vardır:

1. **Three points of contact (üç temas noktası):** *Üç uzvunu sabit tutup yalnızca birini hareket ettiren bir tırmanıcı gibi, aynı anda yalnızca tek bir şeyi değiştirir, diğer her şeyi sabit tutarız. Değişiklik test edilip kararlı hâle gelince sıradakini değiştiririz. Değişiklik olumsuz sonuç verirse geri dönmek kolaydır ve sorunun hangi değişiklikten çıktığını biliriz.*
2. **Copy before release (yayından önce kopyala):** *Yeni bir sürüm yayımlanmadan (değişiklik canlıya alınmadan) önce mevcut kararlı sürümün kopyası/yedeği alınır; yeni sürüm başarısız olursa kararlı kopyaya geri dönülür (baseline, sürüm kontrolü).*
3. **Iterative and incremental software development (yinelemeli ve artırımlı geliştirme):** *Ürün artırımlar (mini projeler) hâlinde geliştirilir ve her artırımda iş akışları yinelenir. Her artırımın sonunda çalışan ama eksik bir ürün, yani kararlı bir ara form vardır. En önemli çekirdekten başlanır (stepwise refinement, Miller yasası) ve ürün kartopu gibi büyütülür (snowball effect); riskler azalır, ürün yeni bilgilerle evrilir.*

2.2 Soru 2 – ISO/IEC 25010 tablosu (komple)

Soru 2

ISO/IEC 25010 kalite karakteristiklerini ve alt karakteristiklerini yazınız.

Çözüm.

O yıl "hepsi" istendi (WhatsApp'a göre tablo 20–25 puan civarındaydı): 9 ana başlık ve 40 alt başlığın tamamı yazılmalı.

Karakteristik	Alt karakteristikler
Functional Suitability (İşlevsel uygunluk)	functional completeness, functional correctness, functional appropriateness *(işlevsel tamlık, işlevsel doğruluk, işlevsel yerindelik)*
Performance Efficiency (Performans verimliliği)	time behaviour, resource utilization, capacity *(zaman davranışı, kaynak kullanımı, kapasite)*
Compatibility (Uyumluluk)	co-existence, interoperability *(birlikte var olabilme, birlikte çalışabilirlik)*
Interaction Capability (Etkileşim yeteneği)	appropriateness recognizability, learnability, operability, user error protection, user engagement, inclusivity, user assistance, self-descriptiveness *(uygunluğun tanınabilirliği, öğrenilebilirlik, çalıştırılabilirlik, kullanıcı hatasından koruma, kullanıcı katılımı, kapsayıcılık, kullanıcıya yardım, kendini açıklayabilirlik)*
Reliability (Güvenilirlik)	faultlessness, availability, fault tolerance, recoverability *(hatasızlık, erişilebilirlik (hazır bulunma), hata toleransı, kurtarılabilirlik)*
Security (Güvenlik)	confidentiality, integrity, non-repudiation, accountability, authenticity, resistance *(gizlilik, bütünlük, inkâr edilemezlik, hesap verebilirlik, kimlik doğruluğu, saldırıya direnç)*
Maintainability (Bakım yapılabilirlik)	modularity, reusability, analysability, modifiability, testability *(modülerlik, yeniden kullanılabilirlik, analiz edilebilirlik, değiştirilebilirlik, test edilebilirlik)*
Flexibility (Esneklik)	adaptability, scalability, installability, replaceability *(uyarlanabilirlik, ölçeklenebilirlik, kurulabilirlik, yer değiştirebilirlik)*
Safety (Emniyet)	operational constraint, risk identification, fail safe, hazard warning, safe integration *(operasyonel kısıt, risk tanımlama, arızada güvenli duruma geçme, tehlike uyarısı, güvenli bütünleşme)*

Türkçe karşılıklar yalnızca anlamak içindir; sınavda maddeleri **İngilizce** yazın. Açıklama istenmedi, maddeler yeterli. 2011 sürümünü bilen biri *Usability* ve *Portability* yazarsa yine puan alabilir, ama hoca 2023 tablosunu kullanıyor.

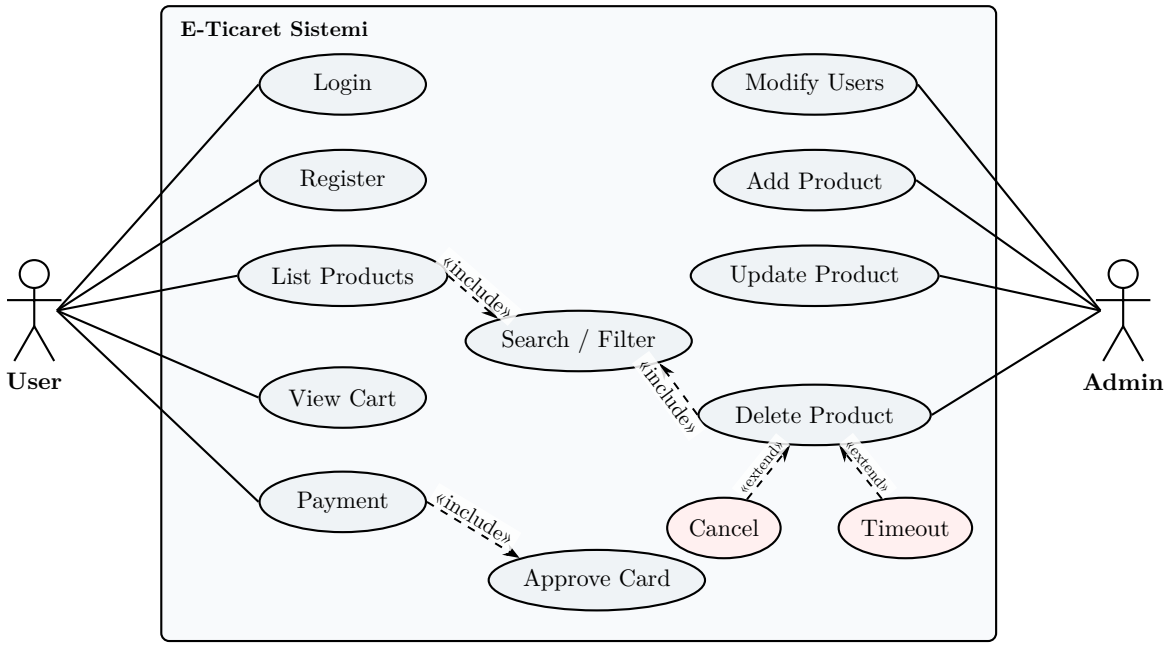
2.3 Soru 3 – E-ticaret use case

Soru 3

Bir alışveriş sitesinde kullanıcı ürün arayabilir/filtreleyebilir, sepete ekleyebilir, stokta varsa ödeme yaparak satın alabilir; yönetici ürün ve kullanıcıları yönetir. Use case diyagramını çiziniz ve use case'leri (brief description, pre/post condition, step-by-step, special cases) açıklayınız.

Çözüm.

Hocanın vizeden sonra tahtada gösterdiği çözüm:



- Aktörler: **User**, **Admin** (gerekirse ödeme sistemi de insan olmayan aktör).
- User: Login, Register, List Products («include» Search/Filter), View Cart (sepete ekleme dahil), Payment («include» Approve Card).
- Admin: Modify Users, Add Product, Update Product, Delete Product («include» Search/Filter); Cancel ve Timeout «extend» Delete Product.

Delete Product (hocanın açıklaması):

- **Unique name (ad):** Delete Product
- **Brief description (kısa tanım):** Yöneticinin ürün silme işlemi anlatılır.
- **Participating actor (aktör):** Admin (yönetici)
- **Pre-condition (ön koşul):** Ürün bulunmuştur.
- **Post-condition (son koşul):** Ürün silinmiştir.
- **Step-by-step description (adımlar):** 1. Yönetici ürünü sistemde arar. 2. Ürünün detayları gösterilir. 3. Yönetici silmeyi onaylar. 4. Ürün 5 ms içinde silinir.
- **Special cases (özel durumlar):** 1. Zaman aşımı (time out). 2. Vazgeçme (cancel).

Payment:

- **Brief description (kısa tanım):** Kullanıcı sepetteki ürünlerin ödemesini yapar.

- **Participating actors (aktörler):** User (modellenirse banka / kredi kartı sistemi insan olmayan aktör olarak)
- **Pre-condition (ön koşul):** Kullanıcı giriş yapmıştır, sepet boş değildir ve ürünler stoktadır.
- **Post-condition (son koşul):** Ödeme tamamlanmış, sipariş oluşturulmuş ve stok azaltılmıştır.
- **Steps (adımlar):** 1. Kullanıcı sepeti görüntüler ve ödemeyi seçer. 2. Kart bilgilerini girer. 3. Sistem kartı onaylar (Approve Card, include). 4. Sistem siparişi oluşturur ve onay gösterir.
- **Special cases (özel durumlar):** kart onaylanmadı; ürün stokta yok; kullanıcı vazgeçti.

Add to Cart açıklaması (soruda vurgulanan işlem):

- **Brief description (kısa tanım):** Kullanıcı seçtiği ürünü alışveriş sepetine ekler.
- **Actor:** User (kullanıcı). **Pre-condition (ön koşul):** Kullanıcı giriş yapmıştır ve ürün listelenmiştir. **Post-condition (son koşul):** Ürün sepettedir ve sepet toplamı güncellenmiştir.
- **Steps (adımlar):** 1. Kullanıcı ürünleri arar/listeler. 2. Kullanıcı bir ürün ve adet seçer. 3. Sistem stoğu kontrol eder. 4. Sistem ürünü sepete ekler. 5. Sistem güncellenmiş sepeti gösterir.
- **Special cases (özel durumlar):** ürün stokta yok (extend: Out of Stock); istenen adet stoktan fazla; kullanıcı vazgeçer.

Dikkat edilecekler: (1) Aktörler sistem dikdörtgeninin **dışında**, use case'ler **içinde**. (2) Aktör–use case arası **düz çizgi**. (3) Her include/extend okunun üzerine stereotip yazılır, yönler doğru çizilir. (4) Hoca kısıtı step-by-step'e somut yazmayı seviyor ("in 5 ms").

2.4 Soru 4 – Cohesion/coupling: tür ve daha iyi hâle getirme

Soru 4 (kod hatırlanmıyor; hocanın derste aynı mantıkla verdiği örnek)

```
void Yazdir(bool isDoc)
{
    if (isDoc) UsePrinter("network");
    else       UsePdfWriter(new int[] {1, 2, 3, 4, 5});
}
void UsePrinter(string b)  { Console.WriteLine(b); }
void UsePdfWriter(int[] a) { Console.WriteLine(a[0] + a[a.Length - 1]); }
void WriteTimePrint()      { Console.WriteLine(DateTime.Now); Print(true); }
```

Modüllerin cohesion türlerini ve aralarındaki coupling'i yazınız; daha iyi cohesion/coupling ile yeniden yazınız.

Çözüm.

- `UsePrinter`: **functional** (tek iş). `UsePdfWriter`: **functional**.
- `WriteTimePrint`: **coincidental** (zaman yazdırma ile yazıcıya gönderme ilgisiz) ya da **temporal** (art arda yapıldıkları için bir arada) — hoca ikisini de kabul etti.
- `Yazdir`: bayrağa göre iki ilgili işten birini seçer ⇒ **logical**; onu `isDoc` bayrağıyla çağıran modül ile arasında **control coupling**.
- `Yazdir` → `UsePdfWriter`: bütün dizi gönderiliyor, yalnızca ilk ve son eleman kullanılıyor ⇒ **stamp coupling**. `UsePrinter("network")`: basit veri ⇒ **data coupling**.

Düzeltilmiş hâli:

```
interface IYazici { void Yaz(); }                          // OCP: yeni cikti turu = yeni sinif
class AgYazici : IYazici { public void Yaz() { Console.WriteLine("network"); } }
class PdfYazici : IYazici
{
    private int ilk, son;
    public PdfYazici(int ilk, int son) { this.ilk = ilk; this.son = son; } // sadece gereken
    veri
    public void Yaz() { Console.WriteLine(ilk + son); }
}
void ZamaniYaz()   { Console.WriteLine(DateTime.Now); } // functional
void Yazdir(IYazici y) { y.Yaz(); }                        // polymorphism, bayrak yok
```

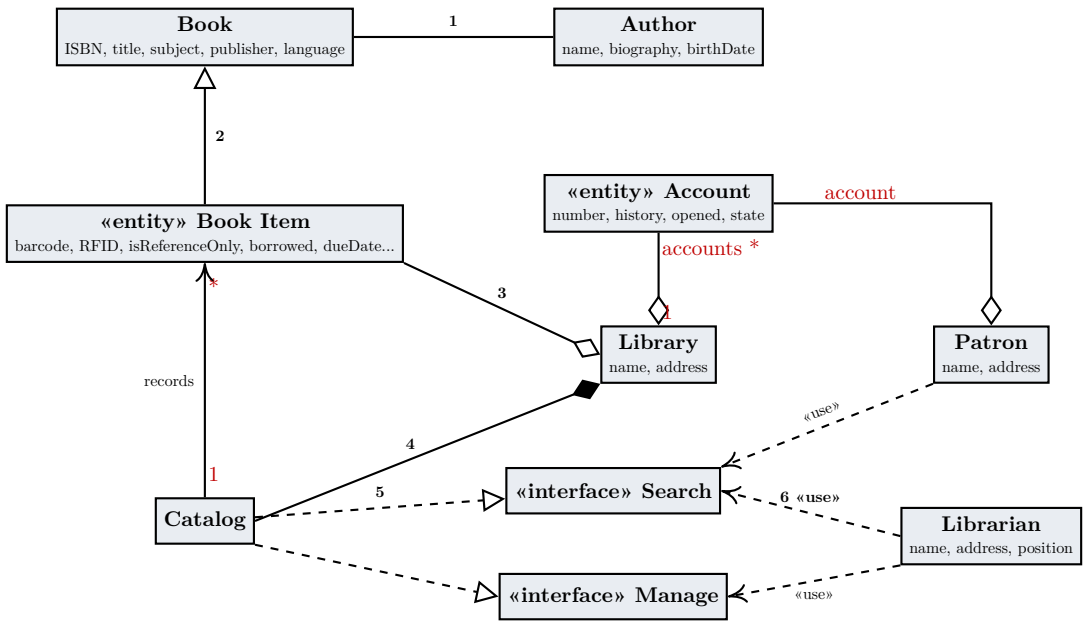
Her metot functional, sınıflar informational; bayrak kalktığı için control coupling, gereken veri gönderildiği için stamp coupling ortadan kalktı (data coupling).

3 Geçmiş yıl sınavı: sınıf diyagramındaki ilişkileri yorumlama (fotoğraf, yılı belirsiz)

Kaynak

WhatsApp grubunda (12.11.2025) paylaşılan sınav kâğıdı fotoğrafı: *"1. Aşağıdaki şekilde verilen ilişkilerin tipini ve senaryodaki sınıflar açısından ne anlama geldiğini aşağıdaki tabloya yazınız. (Her biri 5P, Toplam 25 P)"* Diyagram uml-diagrams.org'daki *Library Domain Model*'dir. Hoca 2025'te de *"class diyagramı verip aralarındaki ilişkileri yorumlamanızı isteyebilirim ama detaylı isterim"* dedi.

Library Domain Model (çizim fotoğraftan yeniden üretildi)



Numaralı ilişkilerin türünü ve sınıflar açısından anlamını tabloya yazınız.

Çözüm.

No	İlişki türü	Açıklama
1	Association	Book ve Author sınıfları birbirinden haberdardır: bir kitap yazar(lar) tarafından yazılır, bir yazarın kitapları vardır; her biri diğerinin metotlarını çağırabilir (binary association, düz çizgi).
2	Inheritance (generalization)	Book Item *bir* Book'tur: kitap kopyası kitabın fiziksel bir örneğidir; ISBN, title, subject, publisher… özelliklerini Book'tan miras alır ve kendi özelliklerini (barcode, RFID, dueDate…) ekler. Boş üçgen üst sınıf Book'u gösterir.
3	Aggregation	Library ◊ Book Item: kütüphane kitap kopyalarından oluşan bir topluluk içerir (topluluk–üye). Kitap kopyası kütüphane olmadan da var olabilir (ödünç verilebilir, transfer edilebilir); yaşam döngüsü kontrolü yoktur. Boş elmas Library tarafında.
4	Composition	Library ♦ Catalog: katalog tam olarak bir kütüphanenin parçasıdır ve onsuz var olamaz; kütüphane kataloğunu oluşturur ve yok eder (yaşam döngüsü kontrolü). Dolu elmas Library tarafında.
5	Realization	Catalog - - ▷ «interface» Search: Catalog sınıfı Search arayüzünde tanımlanan işlemleri gerçekleştirir (Manage arayüzünü de gerçekleştirir).
6	Dependency («use»)	Librarian - - > Search: kütüphaneci Search arayüzünü kullanır (ör. kitap bulmak için); Search değişirse Librarian etkilenebilir. Patron da Search'ü, Librarian Manage'i kullanır.

Ek yorumlar ("detaylı isterim"): Library ◊ Account: **bir** kütüphanenin **çok sayıda** hesabı vardır; Patron ◊ Account: her okuyucunun bir hesabı vardır; Catalog → Book Item (*records*): katalog **çok sayıda** kitap kopyasını kaydeder, tek yönlüdür; «enumeration» Language, AccountState, Format ve «dataType» Address, FullName nitelik tipleri olarak kullanılır.

4 Online dönem test vizesi (çoktan seçmeli, docx)

Kaynak

`oop_ckms_sorular(test).docx` içindeki "VİZEDE ÇIKAN SORULAR" bölümü. Hoca Hafta 9 videosunda o yılın vizesi için *"10 soru var, 40 dakika olacak"* demişti. Biçim değişti (artık klasik), ama konular aynı.

Test 1

Aşağıdaki modül hangi cohesion seviyesine sahiptir?

```
void metot2(int opCode)
{
    if (opCode == 3)      PrintLocalSalesReport();
    else if (opCode == 4) PrintRegionalSalesReport();
    else if (opCode == 5) PrintNationalSalesReport();
    else                  PrintInternationalSalesReport();
}
```

a) Functional b) Procedural c) Communicational d) Temporal e) Logical

Çözüm.

e) Logical cohesion. Modül ilgili bir dizi eylem (dört rapor) yapar ve hangisinin çalışacağını çağıran modül `opCode` ile seçer. Çağıran ile arasında **control coupling** vardır.

Test 2

metot1 ile metot2 arasında hangi tür coupling vardır?

```
void metot1()
{
    int[] a = new int[10];
    for (int i = 0; i < 10; i++) a[i] = i * i;
    int s = metot2(a);
    Console.WriteLine(s);
}
```

```
int metot2(int[] dizi) { return dizi[0] + dizi[9]; }
```

a) Stamp b) Data c) Content d) Control e) Common

Çözüm.

a) Stamp coupling. Bir veri yapısı (10 elemanlı dizi) gönderiliyor ama `metot2` yalnızca `dizi[0]` ve `dizi[9]`'u kullanıyor; gereğinden fazla veri geçiyor. Hoca: *"Dizi gönderiyorsunuz ama dizinin bütün elemanları kullanılmıyor, sadece bazı elemanları; bu da stamp coupling."* Düzeltme: `int metot2(int ilk, int son)` ⇒ data coupling.

Test 3

Hangisi Unified Process'in bir fazı **değildir**? a) Elaboration b) Construction c) Testing d) Transition e) Inception

Çözüm.

c) Testing. Fazlar: inception, elaboration, construction, transition. Test bir *workflow*'dur.

Test 4

Hangileri bakım türleridir? (i. Perfective, ii. Corrective, iii. Adaptive, iv. Reliable) a) I ve IV b) Yalnız I c) I, II ve III d) Yalnız II e) I, III ve IV

Çözüm.

c) I, II ve III. "Reliable" bir bakım türü değildir.

Test 5

Hangisi sağlam (robust) bir mimari sağlar? a) Loose coupling / high cohesion b) High coupling / high cohesion c) Low coupling / low cohesion d) Tight coupling e) Loose coupling / low cohesion

Çözüm.

a) Loose (low) coupling / high cohesion.

Test 6

Modern bakımın doğru tanımını seçiniz. a) Geliştirme ve bakım b) Geliştirme, kurulum ve bakım c) Yazılım her değiştirildiğinde bakım olur d) Geliştirme, bakım ve geliştirme e) Geliştirme, yeniden kullanım

Çözüm.

c) ISO/IEC tanımı: *maintenance occurs whenever software is modified*, kurulumdan önce ya da sonra fark etmez. (a ve b klasik *development-then-maintenance* modelidir.)

Test 7

Yazılım geliştirmede hangisi ayrı bir faz **değildir**? a) Implementation b) Analysis c) Design d) Documentation e) Requirement

Çözüm.

d) Documentation. Dokümantasyon (testing ve planning gibi) bütün fazlarla paralel yürütülür.

Test 8

Hangisi bir ISO/IEC 25010 kalite karakteristiği **değildir**? a) Reliability b) Portability c) Compatibility d) Comprehensibility e) Security

Çözüm.

d) Comprehensibility. (Portability 2011 sürümünde vardır; 2023'te Flexibility adını aldı.)

5 2014–2015 Vize I (sınav kâğıdı fotoğrafı, Bahriye Akay, 27.11.2014)

Kaynak

`Çıkmışlar/WhatsApp Image 2021-11-22.jpg` ve `OOP Çıkmış Sorular.pdf` sayfa 25. Eski vize notundaki *"sorarım demiyorum, soracağım diyorum"* ifadeleri bu sınavın sorularıyla örtüşüyor.

5.1 Soru 1 – Unified Process (15 p)

Soru 1

Unified Process nedir? Fazlarını ve iş akışlarını açıklayınız.

Çözüm.

Unified Process, Booch, Jacobson ve Rumbaugh'un 1999'da yayımladığı nesne yönelimli bir yazılım geliştirme metodolojisidir. Bir adımlar dizisi değildir; her ürüne göre değiştirilen uyarlanabilir bir metodolojidir. UML kullanan bir modelleme tekniğidir; yinelemeli, artırımlı ve iki boyutludur (iş akışları = teknik bağlam, fazlar = iş bağlamı).

İş akışları (workflows):

- **Requirements:** müşterinin ihtiyaçlarını belirle: alanı anla, iş modelini kur, kısıtları belirle (teslim tarihi, paralel çalışma, taşınabilirlik, güvenilirlik, yanıt süresi, maliyet).
- **Analysis:** gereksinimleri tasarımcılar için kesin bir dille analiz et ve incelt; specification document; müşteri onayından sonra SPMP (maliyet ve süre kestirimi, teslim edilecekler, kilometre taşları, bütçe).
- **Design:** analizi gerçekleştirilebilir hâle gelene kadar incelt; mimari tasarım (modüller/sınıflar) ve ayrıntılı tasarım (veri yapıları, algoritmalar); dil seçimi, yeniden kullanım, taşınabilirlik.
- **Implementation:** ürünü seçilen dilde gerçekleştir (alt sistemler, bileşenler).
- **Test:** her geliştiricinin ve SQA grubunun sorumluluğu; izlenebilirlik; gözden geçirmeler; unit, integration, product ve acceptance testleri; alfa ve beta sürümleri.

Fazlar (phases):

- **Inception:** ürün ekonomik olarak yapılabilir mi? Alanı anla, iş modelini kur, kapsamı daralt, ilk business case; üç risk kategorisini belirle (teknik, gereksinim, mimari). Çıktılar: ilk domain/business/requirements modelleri, ilk risk listesi, elaboration planı, ilk business case.
- **Elaboration:** gereksinimleri ve mimariyi incelt, riskleri izle, business case'i incelt, proje yönetim planını hazırla.
- **Construction:** ilk işletme kalitesindeki sürümü (beta release) üret; ağırlık gerçekleştirme ve testte; kullanıcı kılavuzları.

- **Transition:** müşterinin gereksinimlerinin karşılandığından emin ol; hataları düzelt, kılavuzları tamamla, fark edilmemiş riskleri bul; beta sitelerinden gelen geri bildirimle yürür.

5.2 Soru 2 – IEEE SPMP başlıkları (15 p)

Soru 2

IEEE'nin yazılım proje yönetim planının başlıklarını yazınız.

Çözüm.

Hoca yalnızca 8 ana başlığı ister:

1. **Overview** (proje özeti: amaç, kapsam ve hedefler; kabuller ve kısıtlar; teslim edilecekler; zamanlama ve bütçe özeti; planın evrimi)
2. **Reference materials** (referans materyaller)
3. **Definitions and acronyms** (tanımlar ve kısaltmalar)
4. **Project organization** (dış arayüzler, iç yapı, roller ve sorumluluklar)
5. **Managerial process plans** (başlangıç, iş, kontrol, risk yönetimi ve proje kapanış planları)
6. **Technical process plans** (süreç modeli; yöntemler, araçlar ve teknikler; altyapı; ürün kabulü)
7. **Supporting process plans** (konfigürasyon yönetimi, test, dokümantasyon, kalite güvencesi, gözden geçirme ve denetim, problem çözme, alt yüklenici yönetimi, süreç iyileştirme)
8. **Additional plans** (ek planlar)

5.3 Soru 3 – Cohesion ve coupling (15 p)

Soru 3

Kohezyon ve coupling'i açıklayınız. Türlerinin başlıklarını yazınız.

Çözüm.

Cohesion bir modülün içindeki etkileşimin derecesidir; coupling iki modül arasındaki etkileşimin derecesidir. İyi bir tasarımda modül içi etkileşim en yüksek (high cohesion), modüller arası etkileşim en düşüktür (low coupling); bu bakımı kolaylaştırır ve regression fault'ları azaltır.

Cohesion (kötüden iyiye)	Coupling (kötüden iyiye)
1. Coincidental	1. Content
2. Logical	2. Common
3. Temporal	3. Control
4. Procedural	4. Stamp
5. Communicational	5. Data
6. Functional	
7. Informational	

Her seviyeye birer cümlelik tanım eklemek puanı garantiler:

- **Coincidental:** modül birbiriyle tamamen ilgisiz birden çok eylem yapar.
- **Logical:** modül birbiriyle ilgili bir dizi eylem yapar; hangisinin çalışacağını çağıran modül seçer (kontrol bayrağı).

- **Temporal:** modül zaman bakımından ilişkili bir dizi eylem yapar (ör. başlangıç işlemleri).
- **Procedural:** eylemler ürünün izlediği prosedürle (iş akışındaki sırayla) ilişkilidir.
- **Communicational:** eylemler prosedürle ilişkilidir ve hepsi aynı veri üzerinde yapılır.
- **Functional:** modül tam olarak tek bir eylem yapar (tek bir amaca ulaşır).
- **Informational:** modül her biri kendi giriş noktasına ve bağımsız koduna sahip birden çok eylem yapar; hepsi aynı veri yapısı üzerindedir (ADT, sınıf).
- **Content coupling:** bir modül diğerinin içeriğine doğrudan başvurur (yerel verisini/kodunu değiştirir, içine dallanır).
- **Common coupling:** iki modül de aynı global veriye erişir.
- **Control coupling:** bir modül diğerine onun ne yapacağını belirleyen bir kontrol öğesi (bayrak, kod) gönderir.
- **Stamp coupling:** parametre olarak bir veri yapısı gönderilir ama çağrılan modül yalnızca bir kısmını kullanır.
- **Data coupling:** her parametre ya basit veridir ya da bütün elemanları çağrılan modülce kullanılan bir veri yapısıdır.

5.4 Soru 4 – Singleton pattern (15 p)

Soru 4

a) Singleton pattern'i açıklayınız. b) Çözümünü bulunuz. c) Kodla örnek veriniz.

Çözüm.

Not

Bugün design pattern'ler vizeden sonra işleniyor (final konusu). 2014'te vizede sorulmuştu; kısa cevabı bilmekte fayda var.

a) *Singleton, bir sınıfın programın yaşam döngüsü boyunca yalnızca tek bir örneği olmasını garanti eder ve bu örneğe global bir erişim noktası sağlar.* b) Constructor `private` yapılır, tek nesne sınıfın içinde `static` bir alanda tutulur, erişim `static` bir metot/property üzerinden verilir; sınıf `sealed` yapılarak kalıtım engellenir.

```
public sealed class Ayarlar
{
    private static Ayarlar nesne;                // tek ornek
    private Ayarlar() { }                        // disaridan new yapilamaz
    public static Ayarlar Nesne
    {
        get
        {
            if (nesne == null) nesne = new Ayarlar();
            return nesne;
        }
    }
}
// kullanim: Ayarlar a = Ayarlar.Nesne;
```

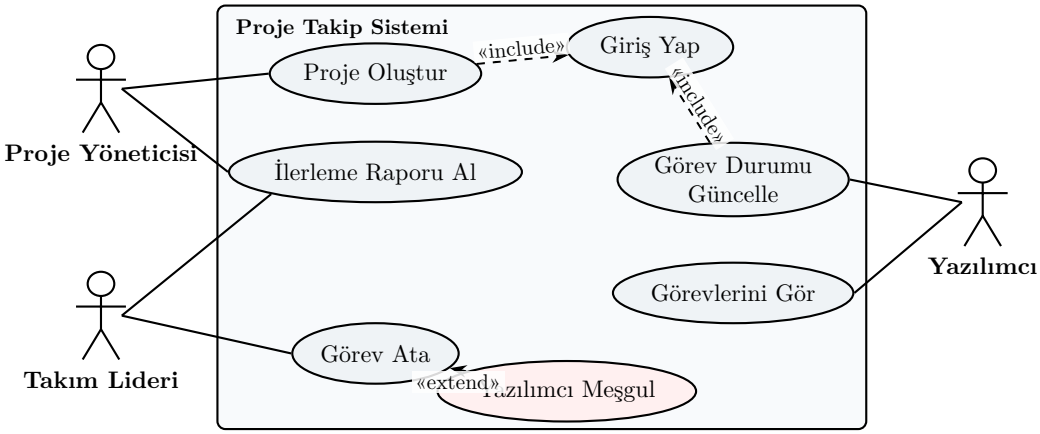
5.5 Soru 5 – Proje takip sistemi (40 p)

Soru 5

Proje yöneticisi, takım lideri ve yazılımcıların olduğu bir proje takip sistemi tasarlanacaktır. a) Use case'lerini oluşturunuz (10). b) Class yapısını oluşturun, class tiplerini (entity, boundary, control) belirtin (10). c) Class'ların tasarımını oluşturunuz (10). d) Collaboration veya sequence diyagramlarından birini seçip oluşturunuz (10).

Çözüm.

a) Use case diyagramı



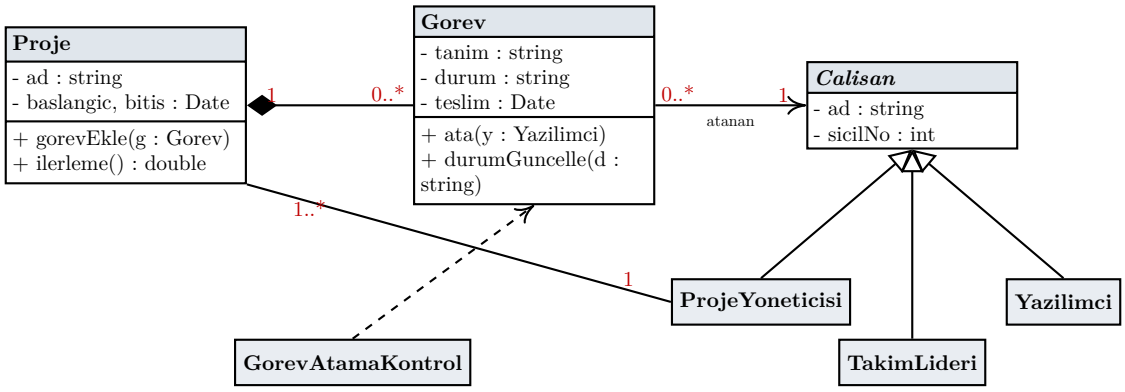
Takım Lideri de ilerleme raporu alabilir. İstenirse ortak bir *Çalışan* aktörü tanımlanıp üç rol ondan **generalization** ile özelleştirilebilir (boş üçgen genel aktöre doğru).

Görev Ata açıklaması: kısa tanım: takım lideri bir projenin görevini bir yazılımcıya atar; aktör: Takım Lideri; ön koşul: proje ve yazılımcı sistemde kayıtlı; son koşul: görev, yazılımcısı ve teslim tarihiyle kaydedilmiş; adımlar: 1. lider projeyi seçer, 2. sistem görevleri ve yazılımcıları listeler, 3. lider bir görev ve yazılımcı seçip teslim tarihi girer, 4. sistem atamayı kaydeder ve yazılımcıya bildirir; özel durumlar: yazılımcı meşgul (extend), lider vazgeçer.

b) Sınıflar ve türleri

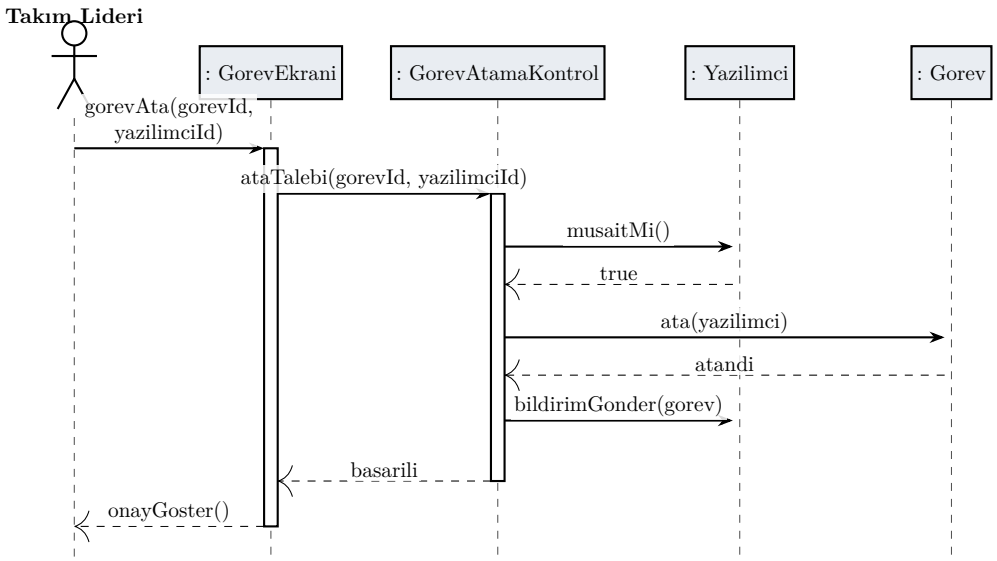
Sınıf	Tür	Gerekçe
Proje, Gorev, Calisan (ProjeYoneticisi, TakimLideri, Yazilimci)	Entity	Uzun ömürlü bilgi tutar.
ProjeEkrani, GorevEkrani, IlerlemeRaporu	Boundary	Giriş/çıkış ekranları ve rapor.
GorevAtamaKontrol, IlerlemeHesaplayici	Control	Atama kuralları ve ilerleme yüzdesi hesabı.

c) Sınıf tasarımı



Proje ♦ Görev composition (görev projesiz yaşamaz); Görev → Çalışan yönlü ilişki (atanan); ProjeYöneticisi, TakımLideri, Yazılımcı Çalışan'dan kalıtılır; kontrol sınıfı görevleri kullanır (dependency).

d) Sequence diyagramı (Görev Ata)



6 Diğer çıkmış ve ödev soruları

6.1 Sınıflar arası ilişki türleri

Kaynak

`Notlar/oop özet.docx` ("OOP 1.) SORU" cevabı) ve WhatsApp'ta (12.11.2025) paylaşılan "OOP 1.) SORU" görseli: altı ilişki çizgisi (Association, Inheritance, Realization, Dependency, Aggregation, Composition) verilip açıklaması istenmiş. Yılı belirsiz.

Soru

Aşağıdaki sınıf ilişkilerini açıklayınız, gösterimlerini çiziniz: a) Association (unary ve binary) b) Inheritance c) Realization d) Dependency e) Aggregation f) Composition.

Çözüm.

a) Association *Unary (tek yönlü):* A sınıfı B'yi bilir ama B, A'dan habersizdir; B'ye (çağrılan sınıfa) bakan açık uçlu okla çizilir. *Binary (çift yönlü):* A, B'yi; B de A'yı bilir (birbirlerinin metotlarını çağırırlar); düz çizgiyle çizilir.

b) Inheritance (generalization) "is-a" (bir ... dır): alt sınıf üst sınıfın nitelik ve işlemlerini miras alır; üst sınıfa bakan **boş üçgenli** düz çizgi.

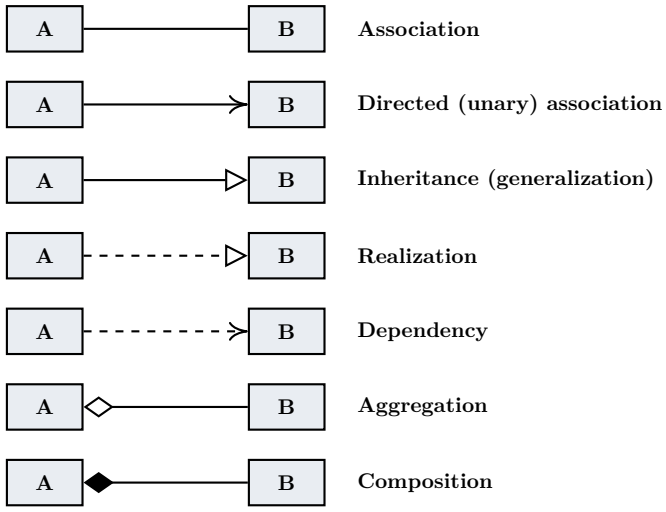
c) Realization Bir sınıfın bir arayüzü gerçekleştirmesi ("can-do"); arayüz yalnızca metot adlarını ve parametrelerini içerir. Arayüze bakan **kesikli çizgi ve boş üçgen**. (Not: öğrenci cevabında "içi dolu ok" yazıyor; bazı çizim araçları üçgeni renkli doldurur ama UML standardında üçgen **boştur**.)

d) Dependency "uses-a" (kullanır): bir öğenin tanımı değişince diğeri etkilenebilir (parametre, yerel değişken); **kesikli çizgi ve açık uçlu ok**.

e) Aggregation "has-a" / topluluk–üye; parça bütün olmadan da var olabilir; **bütün (topluluk) tarafında boş elmas**.

f) Composition En güçlü ilişki; yaşam döngüsü kontrolü olan parça–bütün ilişkisi, parça bütün olmadan var olamaz; **bütün tarafında dolu elmas**.

Gösterimler:



Elmas her zaman **bütünün (collection/whole)** yanına çizilir; kalıtım üçgeni **ata sınıfa**, directed association oku **çağrılan** sınıfa bakar.

C# karşılıkları:

```
class A { B myB; public void DoSomething() { myB.Service(); } }   // binary association:
class B { A myA; public void Operation()   { myA.DoSomething(); } } //   ikisi de birbirini
    cagirir

class C { B myB; public void Do() { myB.Service(); } }   // unary association: C -> B

class Takim { List<Oyuncu> oyuncular;                    // aggregation:
              public void Ekle(Oyuncu o) { oyuncular.Add(o); } } // oyuncu disarida yaratilir

class Ev { List<Oda> odalar = new List<Oda>();           // composition:
           public Ev() { odalar.Add(new Oda()); } }      // parcayi butun yaratir ve yok eder

class Gul : Cicek { }                                    // inheritance (is-a)
interface IUcabilir { void Uc(); }
class Kus : IUcabilir { public void Uc() { } }           // realization (can-do)
class Rapor { public void Bas(Yazici y) { y.Yaz(); } }   // dependency (uses-a)
```

6.2 Adres defteri senaryosu

Kaynak

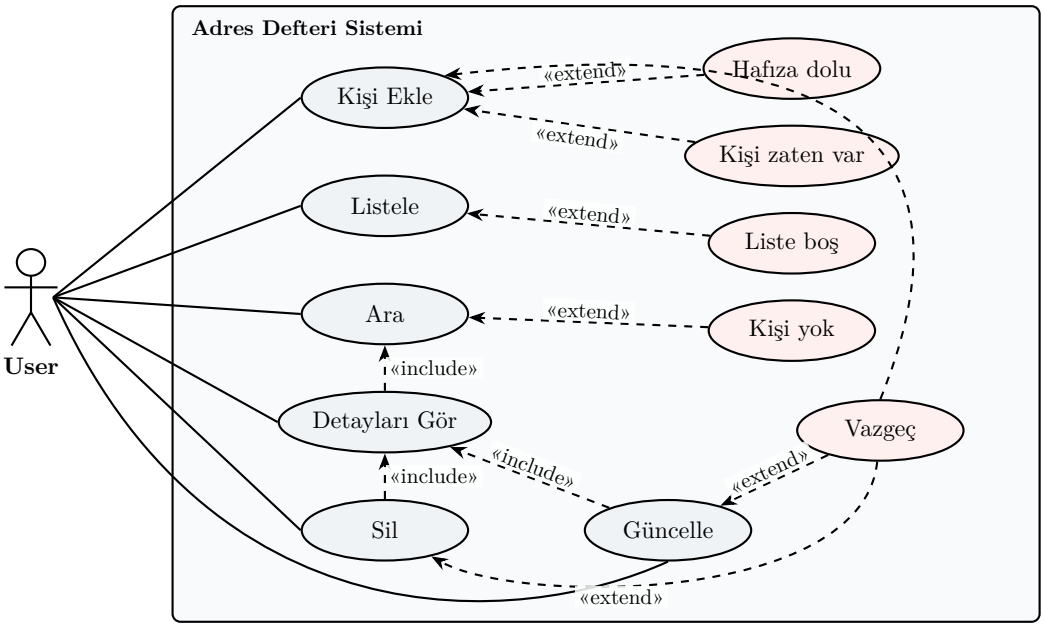
`Çıkmışlar/OOP.docx`: senaryo ve use case açıklama alanları (use-case adı, brief description, entry/exit condition, step-by-step, exceptional cases). Hoca aynı örneği derste use case + sınıf + CRC + collaboration ile tam çözdü.

Soru

Kişilere ait ad, soyad, adres ve telefon bilgisinin tutulduğu bir adres defteri tanımlanmak isteniyor. Adres defterine kişi ekleme, eklenmiş kişide güncelleme yapabilme, silme ve bulma (arama) işlemleri olacaktır. Ayrıca kişileri alfabetik olarak listeleyen bir işlev de gereklidir. Use case'leri (use-case adı, brief description, entry condition, exit condition, step by step description, exceptional cases) ile birlikte modelleyiniz; sınıfları, CRC kartlarını ve bir collaboration/sequence diyagramını çiziniz.

Çözüm.

Use case diyagramı:



Okuma: *Kişi Ekle* her zaman bilgi girişi ister; hafıza doluysa, kişi zaten varsa ya da kullanıcı vazgeçerse istisnai akışlar (**extend**) devreye girer. *Sil* ve *Güncelle* için önce kişinin detayları görülmeli, detay görmek için de önce arama yapılmalı (**include**, her seferinde yapılır).

Kişi Ekle (hocanın tahtadaki açıklaması):

- **Use-case name:** Kişi Ekle (Add Contact)
- **Brief description:** Kullanıcıdan alınan kişi bilgilerinin sisteme kaydedilmesi.
- **Participating actor:** User
- **Pre (entry) condition:** Eklenecek kişinin isim ve soyisim bilgileri bilinmelidir.
- **Post (exit) condition:** Kişi sisteme eklenir.
- **Step-by-step description:** 1. Kişinin isim ve soyismi alınır; varsa adres ve telefon bilgileri alınır. 2. Sisteme kaydedilir.

- **Special cases:** 1. Kişi zaten var. 2. Kullanıcı vazgeçmiş. 3. Hafıza dolu.

Kişi Güncelle:

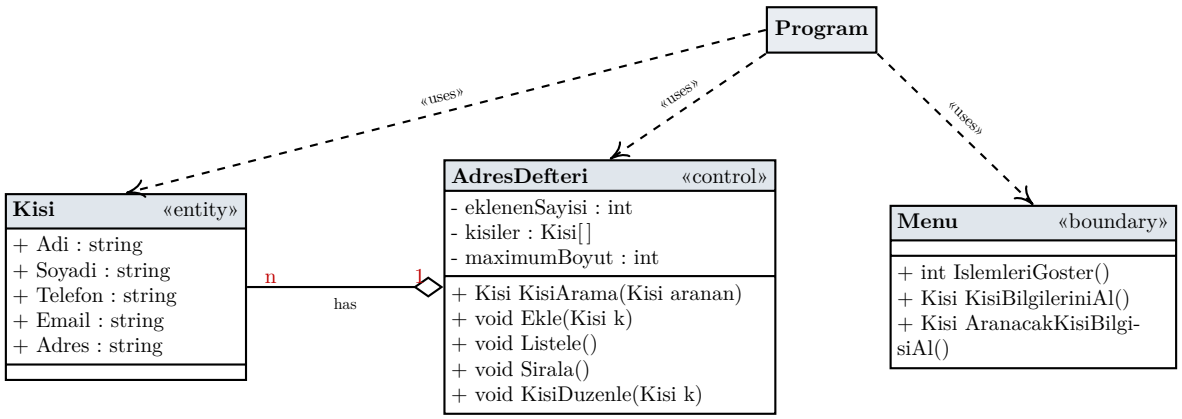
- **Brief description:** Bulunan bir kişinin telefon ve adres bilgisinin güncellenmesi.
- **Entry condition:** Kişinin isim ve soyisim bilgilerinin belirtilmesi.
- **Exit condition:** Adres ve telefon bilgilerinin sistemde güncellenmesi.
- **Step-by-step:** 1. Güncellenecek kişinin isim ve soyismi girilir. 2. Sistemde bu kişi aranır. 3. Var ise yeni adres ve telefon bilgisi girilir. 4. Sistem günceller.
- **Exceptional cases:** Kişi yok; kullanıcı vazgeçti.

Listele: brief: kişilerin alfabetik sırayla listelenmesi; entry: defterde en az bir kişi olmalı; exit: kişiler ekranda alfabetik sırada; steps: 1. kullanıcı listele seçeneğini seçer, 2. sistem kişileri soyada göre sıralar, 3. listeyi gösterir; special: liste boş.

Kişi Sil ve **Kişi Ara**:

- **Kişi Sil:** brief: bulunan bir kişinin defterden silinmesi; entry: silinecek kişinin adı ve soyadı bilinmeli; exit: kişi defterden silinmiş; steps: 1. kullanıcı silmeyi seçer, 2. kişinin adı soyadı girilir, 3. sistem kişiyi arar (include Arama), 4. kişinin detayları gösterilir, 5. kullanıcı onaylar, 6. sistem kişiyi siler; exceptional: kişi yok, kullanıcı vazgeçti.
- **Kişi Ara:** brief: ad ve soyada göre kişinin bulunması; entry: defterde en az bir kişi; exit: kişinin bilgileri gösterilir; steps: 1. ad soyad girilir, 2. sistem kişileri tarar, 3. bulunan kişinin adres ve telefonu gösterilir; exceptional: kişi yok.

Sınıf diyagramı (hocanın kendi çözümü):



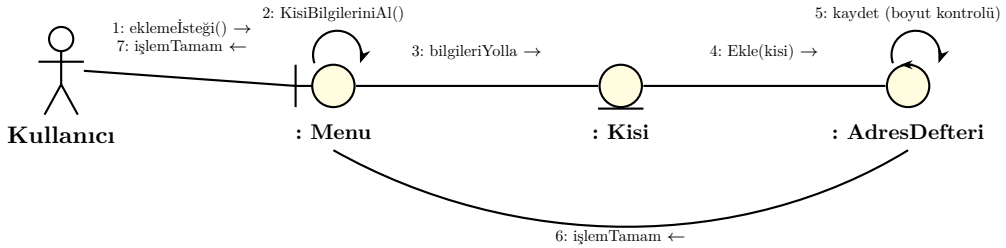
- **Kisi** uzun ömürlü bilgiyi tutar ⇒ entity. **Menu** kullanıcıdan bilgi alır, menüyü gösterir ⇒ boundary. **AdresDefteri** kişiler üzerinde ekleme, arama, sıralama işlemlerini yürütür; hoca kâğıtta bunu **control** olarak işaretledi.
- **AdresDefteri** ◊— **Kisi: aggregation** (*"AdresDefteri has Kişi(ler)"*): bir adres defteri n kişiden oluşan bir topluluk tutar. Elmas AdresDefteri tarafında.
- **Program** diğer sınıfları kullanır (*uses* = dependency).
- Hocanın tahtadaki (daha yeni) sürümünde ise kontrol ayrı bir **DefterController** sınıfına alınmış, AdresDefteri kişileri tutan sınıf olarak kalmış, kullanıcı arayüzü **KullaniciArayuzu** (AnaMenuUret, EkleMenu, SilMenu …) ve **Raporlama** (printRapor) boundary sınıfları eklenmiş; Kisi ile AdresDefteri arası **dolu elmas (composition)** çizilmiş. İki çözüm de geçerlidir: ilişki türünü **gerekçesiyle** yazın (kişiler defterle birlikte silinir ⇒ composition; kişi defterden bağımsız yaşayabilir ⇒ aggregation).

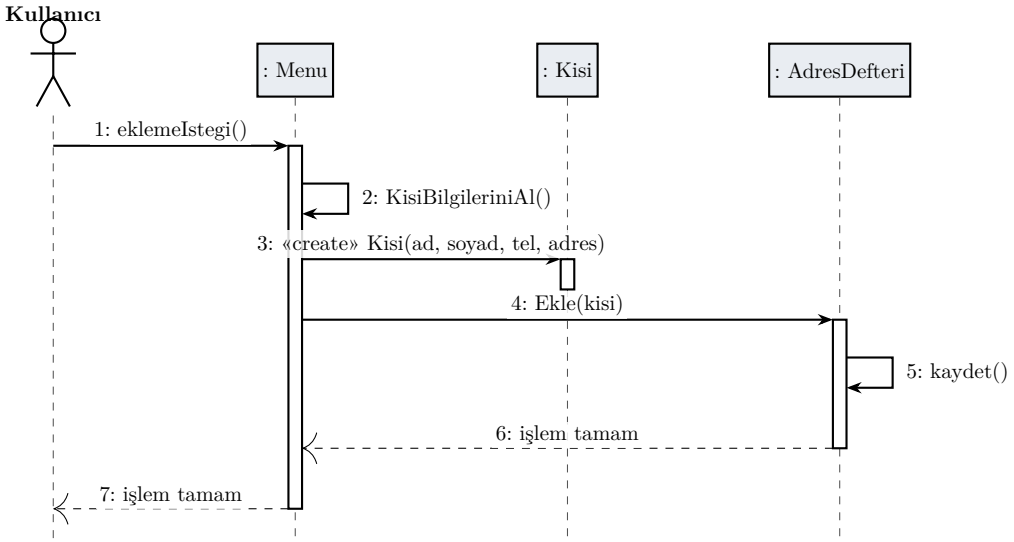
CRC kartları (hocanın kâğıdından):

CLASS: AdresDefteri	CLASS: Menu	CLASS: Program
Responsibility: kişi ekler; kişi siler; kişi düzenler; arama yapar; sıralama yapar	**Responsibility:** ana menüyü gösterir; kişi ekleme bilgilerini alır; kişi arama bilgisini alır	**Responsibility:** menüyü listelemek için Menu nesnesine mesaj yollar; işlemler için AdresDefteri'ne mesaj yollar
Collaboration: Kisi, Program	**Collaboration:** Kisi, AdresDefteri, Program	**Collaboration:** Kisi, Menu, AdresDefteri

Kisi için de: *Responsibility:* kişi bilgilerini tutar ve verir; *Collaboration:* AdresDefteri.

Kişi Ekle: collaboration ve sequence diyagramı:





Akış: kullanıcı ekleme ister (1), menü kişi bilgilerini alır (2, self call), bilgilerle Kisi nesnesi oluşturulur (3), kişi adres defterine eklenmek üzere gönderilir (4), adres defteri maksimum boyutu kontrol edip kaydeder (5, self call), sonuç geriye bildirilir (6, 7). Kapasite doluysa *Kapasite Dolu* extend akışı çalışır (`alt` çerçevesiyle gösterilebilir).

6.3 2021–22 "2. Vize Projesi": Üniversite kütüphane platformu

Kaynak

`Notlar/Project.pdf` (Prof. Dr. Bahriye Akay, 06.12.2021). O yıl ikinci vizenin yerine geçen ödevdi; bugün sınavda gelebilecek senaryo sorularının tam bir örneğidir ve hocanın puanlamasını gösterir: gereksinim iş akışı 25 P (alan anlatımı, sözlük, use case + kısa tanım + adım adım tanım, işlevsel/işlevsel olmayan gereksinimler), analiz iş akışı 55 P (entity/boundary/control sınıfları, CRC ve class diyagramı, statechart, her senaryo için sequence, IEEE SPMP, test planı), tasarım 25 P.

Senaryo

Sistem yöneticileri kitap tanımlar, kitapları öğretim üyelerine ve öğrencilere ödünç verir, iade alır, bu kişilerin bilgilerini düzenler. Öğrenciler ve öğretim üyeleri kitap listesinden kitap seçer, kitap alma talebinde bulunur ve okuduktan sonra iade eder. Gereksinim ve analiz iş akışlarını modelleyiniz.

Çözüm.

Sözlük (glossary) örneği: *Ödünç (loan):* bir kitap kopyasının belirli bir süre için bir üyeye verilmesi. *Üye (member):* öğrenci veya öğretim üyesi. *İade tarihi (due date):* kitabın geri getirilmesi gereken son gün. *Talep (request):* üyenin bir kitabı almak için yaptığı başvuru.

Aktörler: Sistem Yöneticisi; Üye (genel aktör) ← Öğrenci, Öğretim Üyesi (generalization, boş üçgen genele doğru).

Use case'ler: Kitap Tanımla, Üye Bilgisi Düzenle, Ödünç Ver, İade Al (yönetici); Kitap Listele («include» Kitap Ara), Kitap Talep Et (üye); Giriş Yap (hepsi «include»); istisnalar: *Kitap Mevcut Değil* «extend» Kitap Talep Et, *Gecikme* «extend» İade Al.

Gereksinimler: *Functional:* kitap ekleme/silme/düzenleme, üye düzenleme, ödünç verme, iade alma, kitap arama/listeleme, talep. *Nonfunctional:* arama 2 saniyeden kısa sürmeli; aynı anda 200 kullanıcı; web tabanlı çalışmalı; üye bilgileri yetkisiz erişime kapalı (security); haftanın 7 günü erişilebilir (availability).

Sınıflar: Entity: Kitap, KitapKopyasi, Uye (Ogrenci, OgretimUyesi), Odunc, Talep. Boundary: GirisEkrani, KitapListesiEkrani, YoneticiEkrani, GecikmeRaporu. Control: OduncKontrol (ödünç verme kuralları, gecikme hesabı), AramaKontrol.

CRC örneği:

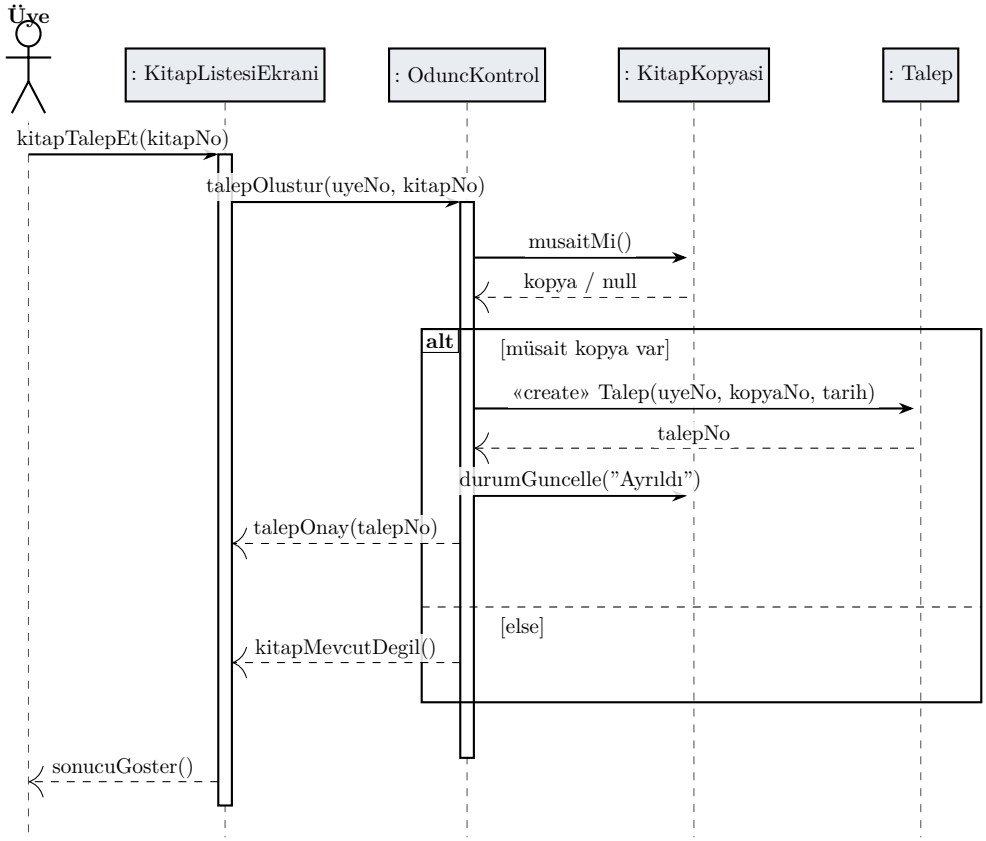
CLASS: OduncKontrol
Responsibility: KitapKopyasi'na müsait olup olmadığını sormak için mesaj gönder; bir Odunc nesnesi oluştur; KitapKopyasi'na durumunu "ödünçte" yapması için mesaj gönder; iade tarihini hesapla; iadede gecikmeyi hesapla
Collaboration: KitapKopyasi, Uye, Odunc, Talep

İlişkiler: Kitap ← KitapKopyasi (bir kitabın birden çok kopyası: Kitap ◊ 1–1..* KitapKopyasi), Uye 1 — 0..* Odunc, Odunc * → 1 KitapKopyasi, Ogrenci/OgretimUyesi → Uye (inheritance).

Statechart (KitapKopyasi): *Rafta* –[ödünç verildi]–> *Ödünçte* –[iade edildi]–> *Rafta*; *Ödünçte* –[süre doldu]–> *Gecikmede* –[iade edildi]–> *Rafta*; *Rafta* –[talep edildi]–> *Ayrıldı* –[ödünç verildi]–> *Ödünçte*.

SPMP (IEEE, 8 ana başlık): 1. Overview, 2. Reference materials, 3. Definitions and acronyms, 4. Project organization, 5. Managerial process plans, 6. Technical process plans, 7. Supporting process plans, 8. Additional plans.

Sequence diyagramı (Kitap Talep Et): üye kitap listesinden bir kitap seçip talep eder; sistem müsait bir kopya arar; kopya **varsa** talep oluşturulur ve kopya "ayrıldı" olarak işaretlenir, **yoksa** "kitap mevcut değil" mesajı gösterilir.



7 Klasik çıkmış kısa sorular bankası

Kaynak

`NESNE YÖNELİMLİ PROGRAMLAMA ÇIKMIŞ SORULAR.docx` (aynı içerik `notlar2.docx` ve el yazısı `OOP Çıkmış Sorular.pdf` sayfa 1–19'da) ve eski "OOP Vize Notlar". Yalnızca vize kapsamındaki sorular alındı; cevaplar kitaba göre düzeltilerek İngilizce/Türkçe verildi.

K1

Yazılım geliştirme yaşam döngüsünün fazlarını yazınız.

Çözüm.

Klasik: requirements, analysis (specification), design, implementation, postdelivery maintenance, retirement. Nesne yönelimli iş akışları: requirements, (OO) analysis, (OO) design, implementation, test (+ postdelivery maintenance, retirement). Planning, testing ve documentation ayrı faz değildir.

K2

Regression fault nedir?

Çözüm.

Yazılımın başka bir yerinde yapılan değişiklik yüzünden, görünüşte ilgisiz bir bölümde ortaya çıkan hata. Information hiding ve düşük coupling regresyon hatasını azaltır; regression testing ile yakalanır.

K3

Verification ve validation nedir?

Çözüm.

Verification: her fazın sonunda yapılan test ("ürünü doğru yapıyor muyuz?"). Validation: projenin sonunda ürünün spesifikasyonu karşılayıp karşılamadığının testi ("doğru ürünü mü yaptık?"). Kitap: ikisi de çok geç; test sürekli yapılmalı.

K4

Moving target problem nedir? Çözümü var mıdır?

Çözüm.

Yazılım ürünü geliştirilirken gereksinimlerin değişmesi. Bağımlılıklara ve regression fault'lara yol açar; çok değişiklik olursa ürün yeniden tasarlanır. **Çözümü yoktur**; iterative-incremental model ve robust mimari etkisini azaltır.

K5

Yineleme (iteration) ve artırım (incrementation) nedir? Miller kuralı ile açıklayınız; yararları nelerdir?

Çözüm.

Miller: insan aynı anda yaklaşık 7 bilgi parçasına odaklanabilir ⇒ stepwise refinement: en önemli kısımlar önce, diğerleri sonra (**incrementation**: ürünü parça parça büyütmek). **Iteration**: her ardışık sürüm hedefe öncekinden yakındır. Yararları: çok sayıda test fırsatı, hatalar erken bulunur; mimarinin sağlamlığı erken belli olur; riskler erken azaltılır; baştan çalışan sürüm olur, müşteri geri bildirimi erken gelir.

K6

Alfa ve beta testleri nelerdir? Sunarken nelere dikkat edilmeli?

Çözüm.

Alpha release: COTS yazılımın geliştirme testleri sürerken olası müşterilere test için verilen sürüm. Beta release: construction fazı sonunda üretilen ilk işletme kalitesindeki sürüm; transition fazı beta sitelerinden gelen geri bildirimle yürür. Alfayı çok erken verip kullanıcının gözünden düşmemeli; betada rakiplerin gerisinde kalmamak için geç kalmamalı.

K7

Riskler kaç kategoriye ayrılır?

Çözüm.

Üç: **technical risks** (teknik riskler), **the risk of not getting the requirements right** (gereksinimleri doğru belirleyememe riski), **the risk of not getting the architecture right** (mimariyi doğru kuramama riski). Riskler sıralanır, kritik olanlar önce azaltılır.

K8

Inception fazının adımları nelerdir?

Çözüm.

1) Alanı anla (understand the domain), 2) iş modelini kur (build the business model), 3) projenin kapsamını daralt (delimit the scope), 4) ilk business case'i hazırlamaya başla (maliyet-etkinlik, yatırımın geri dönüşü, pazar, zamanında teslim, riskler). Amaç: ürün ekonomik olarak yapılabilir mi?

K9

Elaboration, construction ve transition fazlarını açıklayınız.

Çözüm.

Elaboration: başlangıç çıktılarını incelt (gereksinimler, mimari), riskleri izle, business case'i incelt, **project management plan**'ı hazırla. Construction: ilk işletme kalitesinde sürüm (**beta**); implementation, unit/integration/product testing; kullanıcı kılavuzları. Transition: müşteri gereksinimleri karşılanıyor mu, hatalar düzeltilir, kılavuzlar tamamlanır, yeni riskler aranır.

K10

İki boyutlu modele neden ihtiyaç duyulur?

Çözüm.

Gerçek projeler tek seferde tamamlanamayacak kadar büyüktür; Miller kuralı gereği artırımlara (fazlara) bölünür ve her artırımda iş akışları yinelenir. İki boyutlu model hem iş akışlarını (teknik bağlam) hem fazları (iş bağlamı) gösterir; moving target ve kaçınılmaz hatalarla daha iyi başa çıkar, hatalar daha kolay görülür.

K11

Analiz iş akışındaki modeller (modelleme türleri) nelerdir?

Çözüm.

Functional modeling (senaryolar), **entity class modeling** (sınıflar, nitelikler, ilişkiler: class diagram), **dynamic modeling** (her sınıfa yapılan işlemler, durum geçişleri: statechart).

K12

CRC kartı nedir, ne işe yarar, nasıl gösterilir?

Çözüm.

Class–Responsibility–Collaboration: her sınıf için adı, sorumlulukları (ne yapar/bilir) ve işbirliği yaptığı sınıflar yazılır. Sınıfları bulmak (alan bilgisi gerekir) ve özellikle analiz artifact'larını **test** etmek için kullanılır; ekip kartları canlandırınca eksik/hatalı öğeler ortaya çıkar.

K13

Use case nedir? Include ve extends neyi ifade eder?

Çözüm.

Use case, ürün ile aktörleri arasındaki etkileşimi modeller; sistem dikdörtgen, aktör çöp adam, use case elips. «include»: ana use case'in her seferinde kullandığı, yeniden kullanım için ayrılmış davranış (ok ana → kullanılan). «extend»: istisnai/nadir durumlar (ok istisna → ana).

K14

Distribute the control (dağıtık kontrol) nedir?

Çözüm.

Bütün sorumluluğu tek sınıfa yüklemek (God class) yerine kontrolü dağıtmak: asansör probleminde her asansöre ve her kata bir alt kontrolcü ve bir zamanlayıcı (scheduler).

K15

Collaboration diagram nedir?

Çözüm.

Bir senaryonun gerçekleştirilmesinde nesneleri, aralarındaki bağlantıları ve **gönderilme sırasına göre numaralanmış** mesajları gösteren interaction diagram; eşdeğeri sequence diagram'dır.

K16

Specification document nedir, önemi nedir?

Çözüm.

Müşteri ile geliştirici arasında **sözleşmedir**; kesin olmalı, çelişki, eksiklik ve tamamlanmamışlık içermemeli. Test ve bakım için şarttır. Unified Process'te use case'ler ve onlardan türeyen UML artifact'ları bu rolü oynar (müşteri için metin, tasarımcı için diyagram).

K17

Gereksinimler kaç grupta toplanır?

Çözüm.

Functional (ürünün yapması gereken eylemler; girdi–çıktı) ve **nonfunctional** (ürünün özellikleri: platform, yanıt süresi, güvenilirlik; bazıları tasarıma kadar bekler).

K18

CASE aracı nedir? Örnek veriniz.

Çözüm.

Computer-Aided Software Engineering: UML diyagramlarını çizmek, değiştirmek, belgeleri saklamak ve süreci izlemek için yazılım araçları. IBM Rational Rose, Together, ArgoUML, System Architect.

K19

Sınıf türlerini yazarak açıklayınız.

Çözüm.

- **Entity class:** uzun ömürlü bilgiyi modeller (ör. müşteri, kitap, hesap); bilgi ürün kapanınca da saklanır. Stereotip «entity», simgesi altı çizili daire.
- **Boundary class:** ürün ile aktörler (çevresi) arasındaki etkileşimi modeller; genelde girdi/çıktıyla ilgilidir (ekran, form, rapor, yazıcı). Stereotip «boundary», simgesi solunda dikey çizgi olan daire.

- **Control class:** karmaşık hesaplamaları ve algoritmaları modeller; use case akışını yönetir (ör. ücret hesabı, kuralların kontrolü). Stereotip «control», simgesi üstünde ok olan daire.

K20

Information hiding nedir, neden ihtiyaç duyulur?

Çözüm.

Değişmesi muhtemel ayrıntıların modülün içinde gizlenmesi; değişiklik yerelleşir, diğer modüller etkilenmez, regression azalır; kullanıcı kritik verileri doğrudan değiştiremez (en az yetki).

K21

Abstract data type ve abstraction nedir?

Çözüm.

ADT: veri tipi + o tipin örnekleri üzerindeki işlemler (informational cohesion; sınıflar). Abstraction: gereksiz ayrıntıları gizleyip ilgili ayrıntıları vurgulayarak problemi daha üst düzeyde düşünmek (stepwise refinement aracı).

K22

Modül nedir? Bir modülün kaç özelliği vardır?

Çözüm.

Sınır öğeleriyle çevrili, bir toplu tanımlayıcısı (ismi) olan, kodda bitişik program komutları dizisi. Üç özelliği: **function** (ne yapar), **logic** (nasıl yapar), **context** (ne üzerinde çalışır).

K23

Nesne yönelimli programlamanın bakımda sağladığı avantajlar nelerdir?

Çözüm.

Information hiding sayesinde değişiklikler yerel kalır, regression azalır; bağımsız birimler (encapsulation) hatanın yerini bulmayı kolaylaştırır; yeniden kullanım; yüksek cohesion ve düşük coupling'li modüller.

K24

Dynamic binding nedir?

Çözüm.

Hangi metodun çalışacağının nesnenin sınıfına göre **çalışma zamanında** belirlenmesi (`myFile.open()` örneği); polymorphism ile birlikte gelir. Güçlü yandır ama kodu anlamayı zorlaştırabilir.

K25

Görüşme türleri nelerdir? Görüşmeci nasıl olmalı?

Çözüm.

Structured (önceden planlanmış, kapalı uçlu sorular) ve unstructured (cevaplara göre sorulan, açık uçlu sorular). Görüşmeci alanı iyi bilmeli, açık fikirli olmalı; görüşmeden sonra yazılı rapor hazırlayıp bir kopyasını görüşülen kişiye vermeli.

8 Alıştırmalar: koddan cohesion ve coupling bulma

Neden bu bölüm?

Son iki vizede kod verilip tür ve düzeltme istendi, WhatsApp'ta da "kod örneği var mı" diye çok soruldu. Aşağıdaki kodlar sınav tarzındadır.

A1

```
void SistemKapat()
{
    log.Kapat();
    ekran.Temizle();
    yedek.Al();
}
```

Çözüm.

Temporal cohesion: işlemler birbirinden bağımsız, yalnızca kapanış anında art arda yapıldıkları için bir aradadır. Düzeltme: her iş kendi sınıfında (`Logger.Kapat()`, `Ekran.Temizle()`, `Yedekleyici.Al()`); kapanışta çağıran bunları ayrı ayrı çağırır.

A2

```
void Hesapla(char islem, double a, double b)
{
    if (islem == '+') Console.WriteLine(a + b);
    else if (islem == '-') Console.WriteLine(a - b);
    else if (islem == '*') Console.WriteLine(a * b);
}
```

Çözüm.

Logical cohesion; çağıranla **control coupling** (`islem` bir kontrol öğesi). Open-Closed ilkesi ihlali (yeni işlem için `if` eklenir). Düzeltme: `double Topla(double a, double b)`, `Cikar`, `Carp` ayrı metotlar (functional, data coupling) ya da `IIslem` arayüzü ve her işlem için sınıf. (2025–26 *finalinde* aynı tip soru SOLID ile birlikte geldi.)

A3

```
class Ogrenci { public string No, Ad, Soyad, Sifre; public double Ortalama; }
bool BursAlirMi(Ogrenci o) { return o.Ortalama >= 3.0; }
```

Çözüm.

`BursAlirMi` **functional**; çağıranla arasında **stamp coupling** (bütün kayıt, şifre dahil, gönderiliyor; yalnızca `Ortalama` kullanılıyor). Düzeltme: `bool BursAlirMi(double ortalama)` ⇒ data coupling.

A4

```
static class Durum { public static bool indirimVar; }
void KampanyaBaslat() { Durum.indirimVar = true; }
double FiyatHesapla(double f) { return Durum.indirimVar ? f * 0.9 : f; }
```

Çözüm.

İki metot arasında **common coupling** (aynı global veriyi yazıp okuyorlar). Düzeltme: `double FiyatHesapla(double f, double indirimOrani)` (data coupling) ya da kampanya bilgisini ve fiyat hesabını bir `Kampanya` sınıfında kapsülle (information hiding).

A5

```
void NotGirVeOrtalamaGuncelle(Ogrenci o, int not)
{
    o.Notlar.Add(not);
    o.Ortalama = o.Notlar.Average();
}
```

Çözüm.

Communicational cohesion: iki iş (not ekleme, ortalama güncelleme), prosedürle ilişkili ve aynı veri (`o`) üzerinde. Daha iyisi: `Ogrenci` sınıfına `NotEkle(int)` ve `Ortalama()` metotları (sınıf informational, metotlar functional).

A6

```
class Kuyruk
{
    private int[] d = new int[100]; private int bas, son;
    public void Ekle(int x) { d[son++] = x; }
    public int Cikar() { return d[bas++]; }
    public bool BosMu() { return bas == son; }
}
```

Çözüm.

Informational cohesion: her işlemin kendi giriş noktası ve bağımsız kodu var, hepsi aynı (gizli) veri yapısı üzerinde; bir ADT örneği. Düzeltme gerekmez.